

Galaxy Classification Challenge

Astronomy 101 · Week 3 · Computational Module

Topics: Evaluation Metrics · KNN · Gaussian Naive Bayes · SVM

Tools: Python, NumPy, Pandas, scikit-learn

Welcome, astronomer-in-training! Telescopes around the world capture millions of galaxy images every year — far too many for humans to classify by eye. In this assignment, you will build and compare three beginner-friendly machine learning models that classify galaxies into **Spiral**, **Elliptical**, and **Irregular** types using measured physical features instead of raw images. You will also learn why a single number like "accuracy" can hide important mistakes — a lesson real astronomers care about deeply when sorting cosmic objects.

0 Dataset

You are given `week3_galaxy_classification.csv`, a synthetic dataset of **750 galaxies** (250 of each type) generated to resemble real survey measurements. The classes **overlap** — just like in real astronomical surveys — so no model will be perfect. Each row is one galaxy.

Download the dataset here: <https://drive.google.com/file/d/12vJwpnkr1sSfNc1oFuRLbdvAsu7MHZbC/view?usp=sharing>

Column	Meaning	Column	Meaning
<code>galaxy_id</code>	Unique identifier	<code>ellipticity</code>	How "stretched" the galaxy shape is (0 = round)
<code>mean_brightness</code>	Average brightness (mag/arcsec ² ; lower = brighter)	<code>surface_brightness</code>	Light per unit area
<code>galaxy_size</code>	Apparent size in kiloparsecs (kpc)	<code>concentration_index</code>	How concentrated light is near the centre
<code>redshift</code>	Indicates distance / recession velocity	<code>star_formation_rate</code>	New stars formed per year (solar masses/yr)
<code>color_index</code>	Blue (low) vs red (high) galaxy colour	<code>galaxy_type</code>	Target label: Spiral / Elliptical / Irregular

1 Load and Explore the Data

- Load the CSV using Pandas and display the first 10 rows and basic info (`.info()` , `.describe()`).
- Check for missing values and class balance (`value_counts()` on `galaxy_type`).
- Plot at least **two** visualizations (e.g., histogram of `color_index` per class, scatter of `galaxy_size` vs `concentration_index` coloured by class).

2 Prepare and Split the Data

- Separate features (X) from the target label (`galaxy_type`); drop `galaxy_id` .
- Split into **75% training** and **25% testing** sets using `train_test_split` (use `stratify=y` and `random_state=42` for reproducibility).
- Standardize the features using `StandardScaler` .

3 Train Three Classifiers

Train each of the following models on the training set (use default or lightly-tuned hyperparameters — no heavy tuning required):

- **K-Nearest Neighbors** (`KNeighborsClassifier`) — try at least 2 values of `k` (e.g., 3 and 7) and report both.
- **Gaussian Naive Bayes** (`GaussianNB`).
- **Support Vector Machine** (`SVC`)

For each trained model, predict the labels on the **test set**.

4 Evaluate the Models

For every model above, compute and report:

- **Accuracy**
- **Precision, Recall, and F1 Score**
- **Confusion Matrix** — display as a heatmap (e.g., with `seaborn.heatmap` or `matplotlib`)

Use `sklearn.metrics` functions: `accuracy_score` , `precision_score` , `recall_score` , `f1_score` , `confusion_matrix` .

5 Compare Model Performance

- Build a single summary table comparing all models (rows = models, columns = Accuracy, Precision, Recall, F1).
- Make one bar chart comparing F1 scores of the three models side by side.
- Identify which galaxy class is most often **confused** with another, across models, using the confusion matrices.

6 Conclusion

In a markdown cell at the end of your Colab notebook, write a short conclusion (4–5 lines) on which model performed best and why accuracy alone isn't enough to judge a classifier.

Deliverables

1. One Colab Notebook with all code, outputs, and plots clearly labelled.
2. The summary comparison table (Section 5).
3. The written conclusion (Section 6), in a markdown cell.
4. Use only **NumPy**, **Pandas**, **scikit-learn**, and **matplotlib/seaborn** — no need for deep learning libraries.

Tip: Galaxies are messy — real survey data is never perfectly separable. If your model gets 100% accuracy, double-check your code for data leakage (e.g., accidentally including the label as a feature)!